ARTIFICIAL INTELLIGENCE PRACTICAL FILE.

Name:- Mabuoch Bol Manyuon Mabuoch.

Roll no. 20231481.

Submited:- To Ma'am Bhavya

Qn1.

```
tower_of_hanoi(1, Source, Destination, _) :-
    write('Move disk 1 from '), write(Source),
    write(' to '), write(Destination), nl.

tower_of_hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
    tower_of_hanoi(M, Source, Auxiliary, Destination),
    write('Move disk '), write(N),
    write(' from '), write(Source),
    write(' to '), write(Destination), nl,
    tower_of_hanoi(M, Auxiliary, Destination, Source).
```

Output.

```
10 ?- consult(['c:/Users/HP/OneDrive/Desktop/prac.pl']).
true.

11 ?- tower_of_hanoi(3, a, c, b).
Move disk 1 from a to c
Move disk 2 from a to b
Move disk 1 from c to b
Move disk 3 from a to c
Move disk 1 from b to a
Move disk 2 from b to c
Move disk 1 from a to c
true
```

Qn2.

```
move(a,b). move(a,c). move(b,d). move(c,e).


value(a,1). value(b,5). value(c,3).

value(d,8). value(e,2).


hill_climb(Current, Goal) :-

   value(Current,V1),

   findall((V,Next),

      (move(Current,Next), value(Next,V)),

      L),

   sort(L, Sorted),

   reverse(Sorted, [(BestVal,Best)|_]),

   BestVal > V1,

   hill_climb(Best, Goal).


hill_climb(Goal, Goal).
```

Output.

```
26 ?- consult(['c:/Users/HP/OneDrive/Desktop/AI.pl']).
true.

27 ?- hill_climb(a, Goal).
Goal = d
```

Qn3.

```
edge(a,b). edge(a,c). edge(b,d). edge(c,e).

heuristic(a,5). heuristic(b,3).
heuristic(c,4). heuristic(d,1). heuristic(e,2).

best_first([[Node|Path]|_], Node, [Node|Path]).

best_first([[Node|Path]|Rest], Goal, Result) :-
   findall([Next,Node|Path],
      (edge(Node,Next)),
      Children),
   append(Rest, Children, NewPaths),
   sort_paths(NewPaths, Sorted),
   best_first(Sorted, Goal, Result).

sort_paths(Paths, Sorted) :-
   map_list_to_pairs(path_cost, Paths, Pairs),
   keysort(Pairs, SortedPairs),
   pairs_values(SortedPairs, Sorted).

path_cost([Node|_], Cost) :-
   heuristic(Node, Cost).
```

Output.

```
33 ?- consult(['c:/Users/HP/OneDrive/Desktop/pred.pl']).
true.

34 ?- best_first([[a]], Goal, Path).
Goal = a,
Path = [a]
```

Qn4.

```
edge(a,b,1). edge(a,c,3). edge(b,d,3). edge(c,e,2).

heuristic(a,4). heuristic(b,2).
heuristic(c,3). heuristic(d,0). heuristic(e,1).

astar([[Node,Path,G]|_], Node, [Node|Path]).

astar([[Node,Path,G]|Rest], Goal, Result) :-
    findall([Next,[Node|Path],G1],
        (edge(Node,Next,C), G1 is G + C),
        Children),
    append(Rest, Children, NewPaths),
    sort_astar(NewPaths, Sorted),
    astar(Sorted, Goal, Result).

sort_astar(Paths, Sorted) :-
    map_list_to_pairs(f_cost, Paths, Pairs),
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, Sorted).
```

```
f_cost([Node,_,G], F) :-
    heuristic(Node,H),
    F is G + H.
```

Output.

```
true.

37 ?- astar([[a,[],0]], Goal, Path).
Goal = a,
Path = [a]
```

Qn5.

```
% Terminal nodes
terminal(d, 3).
terminal(e, 5).
terminal(f, 2).
terminal(g, 9).

% Tree structure
move(a, b).
move(a, c).
move(b, d).
move(b, e).
move(c, f).
move(c, g).

% Node types
```

```
max_node(a).
min_node(b).
min_node(c).

% Minimax

% If node is terminal → return value
minmax(Node, Value) :-
    terminal(Node, Value), !.

% Max node
minmax(Node, Value) :-
    max_node(Node),
    findall(V, (move(Node,Child), minmax(Child,V)), Values),
    max_list(Values, Value).

% Min node
minmax(Node, Value) :-
    min_node(Node),
    findall(V, (move(Node,Child), minmax(Child,V)), Values),
    min_list(Values, Value).
```

Output.

```
terminal(d, 3).
terminal(e, 5).
terminal(f, 2).
terminal(g, 9).
terminal(d, 3).
terminal(e, 5).
terminal(f, 2).
terminal(g, 9).

true.

53 ?- terminal(Node, Value), !.
Node = d,
Value = 3.
```

Qn6.

```
solve :-
    path((0,0), (4,2), []).


path(State, State, _) :-
    write(State), nl.


path((X,Y), Goal, Visited) :-
    move((X,Y), Next),
    \+ member(Next, Visited),
    path(Next, Goal, [(X,Y)|Visited]),
    write((X,Y)), nl.


move((X,Y),(4,Y)) :- X < 4.
move((X,Y),(X,3)) :- Y < 3.
move((X,Y),(0,Y)) :- X > 0.
move((X,Y),(X,0)) :- Y > 0.
move((X,Y),(X1,Y1)) :-
```

```
    T is min(X,3-Y),
    X1 is X-T, Y1 is Y+T.
move((X,Y),(X1,Y1)) :-
    T is min(Y,4-X),
    Y1 is Y-T, X1 is X+T.
```

Output.

```
54 ?- consult(['c:/Users/HP/OneDrive/Desktop/Qn6.pl']).
true.

55 ?- solve.
4,2
3,3
3,3
3,0
0,3
0,3
4,3
4,0
0,0
true
```

Qn7.

```
sudoku(Rows) :-
    length(Rows,9),
    maplist(same_length(Rows), Rows),
    append(Rows, Vs), Vs ins 1..9,

    maplist(all_distinct, Rows),
    transpose(Rows, Columns),
    maplist(all_distinct, Columns),

    Rows = [A,B,C,D,E,F,G,H,I],
    blocks(A,B,C), blocks(D,E,F), blocks(G,H,I).
```

```
blocks([],[],[]).
blocks([A,B,C|Rest1],[D,E,F|Rest2],[G,H,I|Rest3]) :-
    all_distinct([A,B,C,D,E,F,G,H,I]),
    blocks(Rest1,Rest2,Rest3).
```

Output.

```
56 ?- consult(['c:/Users/HP/OneDrive/Desktop/Qn7.pl']).
true.

57 ?- sudoku(Rows), maplist(writeln, Rows).
[_716,_754,_782,_810,_838,_866,_894,_922,_950]
[_986,_1034,_1082,_1130,_1178,_1226,_1274,_1322,_1370]
[_1418,_1466,_1514,_1562,_1610,_1658,_1706,_1754,_1802]
[_1850,_1898,_1946,_1994,_2042,_2090,_2138,_2186,_2234]
[_2282,_2330,_2378,_2426,_2474,_2522,_2570,_2618,_2666]
[_2714,_2762,_2810,_2858,_2906,_2954,_3002,_3050,_3098]
[_3146,_3194,_3242,_3290,_3338,_3386,_3434,_3482,_3530]
[_3578,_3626,_3674,_3722,_3770,_3818,_3866,_3914,_3962]
[_4010,_4058,_4106,_4154,_4202,_4250,_4298,_4346,_4394]
Rows = [[_A, _B, _C, _D, _E, _F, _G, _H|…], [_J, _K, _L, _M, _
N, _O, _P|…], [_S, _T, _U, _V, _W, _X|…], [_B1, _C1, _D1, _E1,
 _F1|…], [_K1, _L1, _M1, _N1|…], [_T1, _U1, _V1|…], [_C2, _D2|
…], [_L2|…], […|…]],
_A in 1..9,
all_distinct([_A, _B, _C, _J, _K, _L, _S, _T|…]),
all_distinct([_A, _J, _S, _B1, _K1, _T1, _C2, _L2|…]),
all_distinct([_A, _B, _C, _D, _E, _F, _G, _H|…]),
_B in 1..9,
all_distinct([_B, _K, _T, _C1, _L1, _U1, _D2, _M2|…]),
_C in 1..9,
all_distinct([_C, _L, _U, _D1, _M1, _V1, _E2, _N2|…]),
_D in 1..9,
```

```
_A in 1..9,
all_distinct([_A, _B, _C, _J, _K, _L, _S, _T|…]),
all_distinct([_A, _J, _S, _B1, _K1, _T1, _C2, _L2|…]),
all_distinct([_A, _B, _C, _D, _E, _F, _G, _H|…]),
_B in 1..9,
all_distinct([_B, _K, _T, _C1, _L1, _U1, _D2, _M2|…]),
_C in 1..9,
all_distinct([_C, _L, _U, _D1, _M1, _V1, _E2, _N2|…]),
_D in 1..9,
all_distinct([_D, _E, _F, _M, _N, _O, _V, _W|…]),
all_distinct([_D, _M, _V, _E1, _N1, _W1, _F2, _O2|…]),
_E in 1..9,
all_distinct([_E, _N, _W, _F1, _O1, _X1, _G2, _P2|…]),
_F in 1..9,
all_distinct([_F, _O, _X, _G1, _P1, _Y1, _H2, _Q2|…]),
_G in 1..9,
all_distinct([_G, _H, _I, _P, _Q, _R, _Y, _Z|…]),
all_distinct([_G, _P, _Y, _H1, _Q1, _Z1, _I2, _R2|…]),
_H in 1..9,
all_distinct([_H, _Q, _Z, _I1, _R1, _A2, _J2, _S2|…]),
_I in 1..9,
all_distinct([_I, _R, _A1, _J1, _S1, _B2, _K2, _T2|…]),
_J in 1..9,
all_distinct([_J, _K, _L, _M, _N, _O, _P, _Q|…]),
```

```
_K in 1..9,
_L in 1..9,
_M in 1..9,
_N in 1..9,
_O in 1..9,
_P in 1..9,
_Q in 1..9,
_R in 1..9,
_S in 1..9,
all_distinct([_S, _T, _U, _V, _W, _X, _Y, _Z|…]),
_T in 1..9,
_U in 1..9,
_V in 1..9,
_W in 1..9,
_X in 1..9,
_Y in 1..9,
_Z in 1..9,
_A1 in 1..9,
_B1 in 1..9,
all_distinct([_B1, _C1, _D1, _K1, _L1, _M1, _T1, _U1|…]),
all_distinct([_B1, _C1, _D1, _E1, _F1, _G1, _H1, _I1|…]),
_C1 in 1..9,
_D1 in 1..9,
_E1 in 1..9,
```

```
_H1 in 1..9,
all_distinct([_H1, _I1, _J1, _Q1, _R1, _S1, _Z1, _A2|…]),
_I1 in 1..9,
_J1 in 1..9,
_K1 in 1..9,
all_distinct([_K1, _L1, _M1, _N1, _O1, _P1, _Q1, _R1|…]),
_L1 in 1..9,
_M1 in 1..9,
_N1 in 1..9,
_O1 in 1..9,
_P1 in 1..9,
_Q1 in 1..9,
_R1 in 1..9,
_S1 in 1..9,
_T1 in 1..9,
all_distinct([_T1, _U1, _V1, _W1, _X1, _Y1, _Z1, _A2|…]),
_U1 in 1..9,
_V1 in 1..9,
_W1 in 1..9,
_X1 in 1..9,
_Y1 in 1..9,
_Z1 in 1..9,
_A2 in 1..9,
_B2 in 1..9,
```

```
_C2 in 1..9,
all_distinct([_C2, _D2, _E2, _L2, _M2, _N2, _U2, _V2|…]),
all_distinct([_C2, _D2, _E2, _F2, _G2, _H2, _I2, _J2|…]),
_D2 in 1..9,
_E2 in 1..9,
_F2 in 1..9,
all_distinct([_F2, _G2, _H2, _O2, _P2, _Q2, _X2, _Y2|…]),
_G2 in 1..9,
_H2 in 1..9,
_I2 in 1..9,
all_distinct([_I2, _J2, _K2, _R2, _S2, _T2, _A3, _B3|…]),
_J2 in 1..9,
_K2 in 1..9,
_L2 in 1..9,
all_distinct([_L2, _M2, _N2, _O2, _P2, _Q2, _R2, _S2|…]),
_M2 in 1..9,
_N2 in 1..9,
_O2 in 1..9,
_P2 in 1..9,
_Q2 in 1..9,
_R2 in 1..9,
_S2 in 1..9,
_T2 in 1..9,
_U2 in 1..9,
```

```
_S2 in 1..9,
_T2 in 1..9,
_U2 in 1..9,
all_distinct([_U2, _V2, _W2, _X2, _Y2, _Z2, _A3, _B3|…]),
_V2 in 1..9,
_W2 in 1..9,
_X2 in 1..9,
_Y2 in 1..9,
_Z2 in 1..9,
_A3 in 1..9,
_B3 in 1..9,
_C3 in 1..9.
```

Qn8.

male(john). male(paul).

female(mary). female(lisa).

```
parent(john, paul).

parent(mary, paul).

parent(paul, lisa).


father(X,Y) :- male(X), parent(X,Y).

mother(X,Y) :- female(X), parent(X,Y).


grandparent(X,Y) :-

    parent(X,Z), parent(Z,Y).


sibling(X,Y) :-

    parent(Z,X), parent(Z,Y), X \= Y.
```

Output.

```
60 ?- onsult(['c:/Users/HP/OneDrive/Desktop/Qn8.pl']).
Correct to: "consult(['c:/Users/HP/OneDrive/Desktop/Qn8.pl'])"? yes
true.

61 ?- father(X, paul),
|     mother(X, paul),
|      grandparent(X, lisa),
|     sibling(paul, X).
false.

62 ?- father(X, paul),
|     mother(X, paul),
|     grandparent(X, lisa),
|     sibling(paul, X).
false.
```

Qn9.

```
% Define frames

frame(person, [

    name:unknown,

    age:0,
```

```
    gender:unknown
]).

frame(student, [
    isa:person,
    course:bca
]).

% Get slot value (direct)
get(Frame, Slot, Value) :-
    frame(Frame, List),
    member(Slot:Value, List).

% Inheritance: if slot not found, check parent
get(Frame, Slot, Value) :-
    frame(Frame, List),
    member(isa:Parent, List),
    get(Parent, Slot, Value).
```

Output.

```
71 ?- consult(['c:/Users/HP/OneDrive/Desktop/Qn9.pl']).
true.

72 ?- get(student, course, Value).
Value = bca .

73 ?-  get(student, age, Value).
Value = 0 .

74 ?- get(student, gender, Value).
Value = unknown .

75 ?- get(person, name, Value).
Value = unknown
```

Qn10.

```
conc([], L, L).

conc([H|T], L2, [H|R]) :-

   conc(T, L2, R).
```

Output.

```
76 ?- consult(['c:/Users/HP/OneDrive/Desktop/Qn10.pl']).
true.

77 ?- conc([1,2,3], [4,5], L).
L = [1, 2, 3, 4, 5].
```

Qn11.

```
reverse([], []).

reverse([H|T], R) :-

   reverse(T, RT),

   append(RT, [H], R).
```

Output.

```
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

78 ?- reverse([1,2,3,4], R).
R = [4, 3, 2, 1].
```

Qn12.

sentence(s(NP,VP)) --> noun_phrase(NP), verb_phrase(VP).

noun_phrase(np(Det,N)) --> determiner(Det), noun(N).

verb_phrase(vp(V,NP)) --> verb(V), noun_phrase(NP).

determiner(det(the)) --> [the].

noun(n(cat)) --> [cat].

noun(n(dog)) --> [dog].

verb(v(chases)) --> [chases].

Output.

```
79 ?- consult(['c:/Users/HP/OneDrive/Desktop/Qn12.pl']).
true.

80 ?- sentence(Tree, [the,cat,chases,the,dog], []).
Tree = s(np(det(the), n(cat)), vp(v(chases), np(det(the), n(dog)))).
```

Qn13.

s --> [].

s --> [a], s, [b].

Output.

```
81 ?- consult(['c:/Users/HP/OneDrive/Desktop/Qn13.pl']).
true.

82 ?- phrase(s, [a,a,b,b]).
true
```